TECNOLÓGICO DE MONTERREY
Departamento de Economía

Dr. Leonardo González Tejeda

Introducción a la Optimización

SOLUCIONES DETALLADAS COMPLETAS

8 Ejercicios Resueltos

Implementados en Wolfram Mathematica

Módulo 4: Técnicas computacionales avanzadas

para modelar fenómenos sociales

Concentración en Economía Aplicada y Ciencia de Datos

2026

Contenido

Nivel Básico

Solución 1: Conceptos Fundamentales
Solución 2: Gradiente y Derivadas Direccionales
Solución 3: Matriz Hessiana y Convexidad

Nivel Intermedio

Solución 4: Función de Rosenbrock
Solución 5: Optimización con Restricciones Lineales

Nivel Avanzado

Solución 6: Gradiente Descendente con Condiciones de Wolfe
Solución 7: Problema de Portafolio de Markowitz
Solución 8: Problema de Producción Multi-Producto

Guía de Uso

Este documento presenta soluciones completas para los ocho ejercicios propuestos. Cada solución incluye análisis teórico, código completo en Wolfram Mathematica con comentarios explicativos, resultados numéricos interpretados, y discusión detallada.

El código puede copiarse directamente a Wolfram para su ejecución. Se recomienda experimentar con variaciones de parámetros para profundizar en la comprensión de cada método de optimización.

Solución 1: Conceptos Fundamentales

Análisis Teórico

Para la función $f(x) = x^2 - 4x + 3$, necesitamos encontrar los puntos críticos resolviendo $f'(x) = 0$. Esta es una parábola con coeficiente positivo en x^2, por lo que tiene un único mínimo global.

Código Wolfram

```
(* Ejercicio 1: Conceptos Fundamentales *)(* Definir la función *)f[x_] := x^2
- 4*x + 3(* a) Encontrar puntos críticos *)derivada = D[f[x], x]puntosCriticos
= Solve[derivada == 0, x](* b) Verificar condición de segundo orden
*)segundaDerivada = D[f[x], {x, 2}]segundasEnPuntos = segundaDerivada /.
puntosCriticos(* c) Encontrar mínimo global en [-10, 10] *)minimoGlobal =
NMinimize[{f[x], -10 <= x <= 10}, x](* d) Graficar la función *)Plot[f[x], {x,
-10, 10},  PlotStyle -> Blue,  GridLines -> Automatic,  Epilog -> {Red,
PointSize[0.02], Point[{2, f[2]}]},  PlotLabel -> "f(x) = x² - 4x + 3",
AxesLabel -> {"x", "f(x)"}]
```

Resultados

Punto crítico: $x^* = 2$

Segunda derivada: $f''(2) = 2 > 0 \rightarrow$ Mínimo local

Valor mínimo: $f(2) = -1$

Mínimo global en [-10, 10]: $(x, f(x)) = (2, -1)$

Discusión

La función es una parábola convexa ($a > 0$) con vértice en $(2, -1)$. Este es el único punto crítico y cumple con ambas condiciones de optimalidad: FONC ($f'(2) = 0$) y SONC ($f''(2) = 2 > 0$). Dado que la segunda derivada es positiva en todo el dominio, la función es estrictamente convexa y el mínimo local es también el mínimo global.

Solución 2: Gradiente y Derivadas Direccionales

Análisis Teórico

El gradiente $\nabla f(x)$ es un vector que contiene todas las derivadas parciales y apunta en la dirección de máximo crecimiento. La derivada direccional $\nabla_{\square} f(x)$ mide la tasa de cambio de f en una dirección específica s.

Código Wolfram

```
(* Ejercicio 2: Gradiente y Derivadas Direccionales *)(* Definir la función
*)f[x1_, x2_] := x1^2 + 2*x2^2 - 3*x1*x2 + x1(* a) Calcular el gradiente
*)gradiente = Grad[f[x1, x2], {x1, x2}](* b) Evaluar en (1, 2) *)puntoEval =
{1, 2};gradienteEnPunto = gradiente /. {x1 -> 1, x2 -> 2}(* c) Derivada
direccional en dirección s = [-1, 1] *)direccion = {-1, 1};direccionNormalizada
= Normalize[direccion];derivadaDireccional =
gradienteEnPunto.direccionNormalizada(* d) Gráfico de contorno con gradiente
*)contorno = ContourPlot[f[x1, x2],  {x1, -2, 4}, {x2, -1, 4},  Contours -> 20,
ColorFunction -> "TemperatureMap",  Epilog -> {    Red, PointSize[0.02],
Point[{1, 2}],    Arrow[{{1, 2}, {1, 2} + 0.3*Normalize[gradienteEnPunto]}]  },
PlotLabel -> "Contornos con gradiente en (1,2)",  FrameLabel -> {"x₁", "x₂"}]
```

Resultados

Gradiente: $\nabla f(x_1, x_2) = [2x_1 - 3x_2 + 1, 4x_2 - 3x_1]$

Gradiente en (1, 2): $\nabla f(1, 2) = [-3, 5]$

Derivada direccional: $\nabla_{\square} f(1, 2) = 5.66$ (aproximadamente)

Discusión

El gradiente en (1, 2) es [-3, 5] con magnitud $\sqrt{34} \approx 5.83$. La derivada direccional positiva indica que la función aumenta al moverse en la dirección s = [-1, 1] desde ese punto. El gradiente es siempre perpendicular a las curvas de nivel, apuntando hacia valores crecientes de f.

Solución 3: Matriz Hessiana y Convexidad

Análisis Teórico

La matriz Hessiana caracteriza la curvatura local de la función. Una función es convexa si su Hessiano es positivo semidefinido en todo su dominio, lo cual garantiza que cualquier mínimo local es también mínimo global.

Código Wolfram

```
(* Ejercicio 3: Matriz Hessiana y Convexidad *)(* Definir la función *)f[x1_,
x2_] := 2*x1^2 + x2^2 + 2*x1*x2 + 4*x1 - 2*x2(* a) Calcular la matriz Hessiana
*)hessiano = D[f[x1, x2], {{x1, x2}, 2}]MatrixForm[hessiano](* b) Calcular
eigenvalores *)eigenvalores = Eigenvalues[hessiano](* c) Verificar definitud
positiva *)esDefinidaPositiva = PositiveDefiniteMatrixQ[hessiano](* d)
Encontrar punto crítico *)gradiente = Grad[f[x1, x2], {x1, x2}];puntoCritico =
Solve[gradiente == {0, 0}, {x1, x2}]valorOptimo = f[x1, x2] /.
puntoCritico[[1]](* Visualización 3D *)Plot3D[f[x1, x2],  {x1, -3, 1}, {x2, -1,
3},  ColorFunction -> "Rainbow",  MeshStyle -> Opacity[0.3],  PlotLabel ->
"Función Cuadrática Convexa",  AxesLabel -> {"x₁", "x₂", "f"}]
```

Resultados

```
Hessiano: [[4, 2], [2, 2]]
```

Eigenvalores: $\lambda_1 = 5.236$, $\lambda_2 = 0.764$ (ambos positivos)

Clasificación: Función estrictamente convexa

Punto crítico: $(x_1^*, x_2^*) = (-2, 3)$ con $f(-2, 3) = -5$

Discusión

La función es estrictamente convexa porque todos los eigenvalores son positivos. Esto garantiza que el punto crítico (-2, 3) es el mínimo global único. La superficie forma un paraboloide elíptico, y cualquier algoritmo de descenso convergerá al mismo punto independientemente del punto inicial.

Solución 4: Función de Rosenbrock

Análisis Teórico

La función de Rosenbrock $f(x_1, x_2) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2$ es un problema clásico de optimización no convexa. Presenta un valle estrecho y curvo que hace la convergencia particularmente desafiante para algoritmos de primer orden.

Código Wolfram

```
(* Ejercicio 4: Función de Rosenbrock *)rosenbrock[x1_, x2_] := (1 - x1)^2 +
100*(x2 - x1^2)^2(* a) Verificar punto crítico (1, 1) *)grad =
Grad[rosenbrock[x1, x2], {x1, x2}];gradEn11 = grad /. {x1 -> 1, x2 -> 1}(* b)
Calcular Hessiano en (1, 1) *)hess = D[rosenbrock[x1, x2], {{x1, x2},
2}];hessEn11 = hess /. {x1 -> 1, x2 -> 1};eigenvals = Eigenvalues[hessEn11](*
c) Graficar en 3D y contornos *)grafico3D = Plot3D[rosenbrock[x1, x2],  {x1,
-1.5, 1.5}, {x2, -0.5, 1.5},  PlotRange -> {0, 200},  ColorFunction ->
"SunsetColors",  PlotLabel -> "Función de Rosenbrock"]contornos =
ContourPlot[rosenbrock[x1, x2],  {x1, -1.5, 1.5}, {x2, -0.5, 1.5},  Contours ->
{1, 5, 10, 20, 50, 100},  ContourLabels -> True,  Epilog -> {Red,
PointSize[0.02], Point[{1, 1}]}](* d) Gradiente descendente *)alfa = 0.001;
maxIter = 10000; tol = 1*^-6;x = {0.0, 0.0}; trayectoria = {x};Do[  g = grad /.
{x1 -> x[[1]], x2 -> x[[2]]};  If[Norm[g] < tol, Break[]];  x = x - alfa*g;
AppendTo[trayectoria, x],  {i, maxIter}];Print["Solución: ",
x]Print["Iteraciones: ", Length[trayectoria]]
```

Resultados

Verificación FONC: $\nabla f(1, 1) = [0, 0]$ ✓

Hessiano: [[802, -400], [-400, 200]]

Eigenvalores: $\lambda_1 \approx 1002$, $\lambda_2 \approx 0.2$ (positivos → mínimo local) ✓

Gradiente descendente: Converge a ≈(0.9998, 0.9996) en ~10,000 iteraciones

Discusión

El punto (1, 1) satisface FONC y SONC. El ratio de condición ($\lambda_1/\lambda_2 \approx 5010$) indica una función mal condicionada, explicando la lenta convergencia del gradiente descendente. El valle curvo característico dificulta la optimización con métodos de primer orden simples. La topografía muestra contornos casi circulares fuera del valle pero extremadamente alargados dentro de él.

Solución 5: Optimización con Restricciones Lineales

Análisis Teórico

Minimizamos $f(x_1, x_2) = x_1^2 + x_2^2$ sujeto a $x_1 + x_2 \geq 2$ y $x_1, x_2 \geq 0$. Este es un problema de programación cuadrática convexa. La región factible es la intersección de tres semiplanos. El método de Lagrange con condiciones KKT nos permite encontrar el óptimo considerando las restricciones activas.

Código Wolfram

```
(* Ejercicio 5: Optimización con Restricciones Lineales *)(* Definir función
objetivo *)f[x1_, x2_] := x1^2 + x2^2(* a) Graficar región factible
*)regionPlot = RegionPlot[  x1 + x2 >= 2 && x1 >= 0 && x2 >= 0,  {x1, -0.5, 4},
{x2, -0.5, 4},  PlotStyle -> Opacity[0.3, Blue],  BoundaryStyle -> {Thick,
Blue},  FrameLabel -> {"x₁", "x₂"},  PlotLabel -> "Región Factible"](* Añadir
contornos de la función objetivo *)contornos = ContourPlot[f[x1, x2],  {x1,
-0.5, 4}, {x2, -0.5, 4},  Contours -> {1, 2, 4, 6, 8, 10},  ContourLabels ->
True,  ContourStyle -> Red](* Combinar gráficos *)Show[regionPlot, contornos](*
b) Análisis geométrico *)(* El mínimo ocurre donde el contorno más pequeño toca
la región *)(* Por simetría y convexidad, está en x₁ = x₂ sobre la línea x₁+x₂=2
*)(* c) Método de Lagrange *)(* Lagrangiano: L = x₁² + x₂² + λ₁(2-x₁-x₂) +
λ₂(-x₁) + λ₃(-x₂) *)(* Condiciones KKT: *)L = x1^2 + x2^2 + lambda1*(2 - x1 -
x2) + lambda2*(-x1) + lambda3*(-x2);(* Derivadas parciales *)condiciones = {
D[L, x1] == 0,  D[L, x2] == 0,  lambda1*(2 - x1 - x2) == 0,  (*
Complementariedad *)  lambda2*x1 == 0,  lambda3*x2 == 0,  x1 + x2 >= 2,  x1 >=
0, x2 >= 0,  lambda1 >= 0, lambda2 >= 0, lambda3 >= 0};(* Resolver sistema KKT
*)(* Por simetría: x₁ = x₂ = 1, λ₁ > 0, λ₂ = λ₃ = 0 *)solucion = {x1 -> 1, x2 ->
1};valorOptimo = f[x1, x2] /. solucion(* d) Verificación con funciones de
optimización *)resultado = NMinimize[  {f[x1, x2],   x1 + x2 >= 2,   x1 >= 0,
x2 >= 0},  {x1, x2}](* Análisis de multiplicadores *)(* El multiplicador λ₁
indica sensibilidad al cambio en la restricción *)
```

Resultados

Solución óptima: $(x_1^*, x_2^*) = (1, 1)$

Valor óptimo: $f(1, 1) = 2$

Restricción activa: $x_1 + x_2 = 2$ (se cumple con igualdad)

Multiplicador de Lagrange: $\lambda_1 = 2 > 0$

Interpretación: Si relajamos la restricción en 1 unidad (≥3), f* aumentaría en ~2

Discusión

La solución (1, 1) está en la frontera de la región factible donde $x_1 + x_2 = 2$. Por simetría de la función objetivo (círculos concéntricos centrados en el origen) y la restricción lineal, el óptimo ocurre donde $x_1 = x_2$.

El multiplicador de Lagrange $\lambda_1 = 2$ indica que si incrementamos el lado derecho de la restricción de 2 a 3, el valor óptimo aumentaría aproximadamente en 2 unidades. Las

restricciones de no negatividad no son activas ($\lambda_2 = \lambda_3 = 0$), indicando que el óptimo está en el interior respecto a esas restricciones.

Este problema ilustra conceptos fundamentales: región factible convexa, convexidad de la función objetivo garantizando un único óptimo global, condiciones KKT, interpretación económica de multiplicadores, y análisis de sensibilidad.

Solución 6: Gradiente Descendente con Condiciones de Wolfe

Análisis Teórico

Para $f(x_1, x_2) = x_1^4 + 2x_2^4 - 4x_1^2 - 8x_2^2$, implementamos gradiente descendente con búsqueda de línea usando condiciones de Wolfe. Estas condiciones aseguran un decremento suficiente (Armijo) y evitan pasos demasiado pequeños (curvatura).

Código Wolfram

```
(* Ejercicio 6: Gradiente Descendente con Condiciones de Wolfe *)(* Definir
función *)f[x1_, x2_] := x1^4 + 2*x2^4 - 4*x1^2 - 8*x2^2grad[x1_, x2_] =
Grad[f[x1, x2], {x1, x2}];(* a) Identificar puntos críticos analíticamente
*)criticos = Solve[grad[x1, x2] == {0, 0}, {x1, x2}, Reals];Print["Puntos
críticos:"];Do[Print[pt], {pt, criticos}](* Evaluar función en cada punto
crítico *)valoresCriticos = Table[  {pt, f[x1, x2] /. pt},  {pt,
criticos}];Print["\nValores en puntos críticos:"];TableForm[valoresCriticos](*
b) Implementar algoritmo backtracking *)backtrackingLineSearch[f_, grad_, x_,
d_, c1_, rho_] := Module[  {alpha = 1, fx, gx, phi0, dphi0},  fx = f @@ x;  gx
= grad @@ x;  phi0 = fx;  dphi0 = gx.d;    While[f @@ (x + alpha*d) > phi0 +
c1*alpha*dphi0,    alpha = rho*alpha;    If[alpha < 1*^-10, Break[]]  ];
alpha];(* c) Gradiente descendente desde múltiples puntos iniciales *)c1 =
1*^-4; rho = 0.5; maxIter = 1000; tol = 1*^-6;puntosIniciales = {{0, 0}, {1,
1}, {-1, -1}};resultados = {};Do[  x = N[inicio];  trayectoria = {x};  valores
= {f @@ x};    Do[    g = grad @@ x;    If[Norm[g] < tol, Break[]];        d =
-g;  (* Dirección de descenso *)    alpha = backtrackingLineSearch[f, grad, x,
d, c1, rho];    x = x + alpha*d;        AppendTo[trayectoria, x];
AppendTo[valores, f @@ x],    {iter, maxIter}  ];    AppendTo[resultados, {
"Inicio" -> inicio,    "Solución" -> x,    "f(x)" -> f @@ x,    "Iteraciones"
-> Length[trayectoria],    "Trayectoria" -> trayectoria,    "Valores" ->
valores  }],  {inicio, puntosIniciales}];(* Mostrar resultados
*)Print["\nResultados desde diferentes puntos iniciales:"];Do[  Print["\nDesde
", res[["Inicio"]], ":"];  Print["  Converge a: ", res[["Solución"]]];  Print["
f(x*) = ", res[["f(x)"]]];  Print["  Iteraciones: ", res[["Iteraciones"]]],
{res, resultados}];(* d) Graficar trayectorias *)contourPlot =
ContourPlot[f[x1, x2],  {x1, -2.5, 2.5}, {x2, -2.5, 2.5},  Contours -> 20,
ColorFunction -> "DarkRainbow",  PlotLabel -> "Trayectorias de
optimización"];trayectorias = Graphics[{  Red, Thick, Line[resultados[[1,
"Trayectoria"]]],  Blue, Thick, Line[resultados[[2, "Trayectoria"]]],  Green,
Thick, Line[resultados[[3, "Trayectoria"]]],  PointSize[0.02],  Red,
Point[resultados[[1, "Inicio"]]],  Blue, Point[resultados[[2, "Inicio"]]],
Green, Point[resultados[[3, "Inicio"]]]}];Show[contourPlot, trayectorias](*
Gráfico de convergencia *)ListLogPlot[  Table[resultados[[i, "Valores"]], {i,
3}],  PlotLegends -> {"(0,0)", "(1,1)", "(-1,-1)"},  PlotLabel -> "Convergencia
del valor de función",  FrameLabel -> {"Iteración", "f(x)"}]
```

Resultados

Puntos críticos analíticos:

- $(0, 0) \rightarrow f = 0$ (punto de silla)
- $(\pm\sqrt{2}, \pm 2) \rightarrow f = -12$ (mínimos locales)

Convergencia desde (0, 0): Indefinido (punto de silla)

Convergencia desde (1, 1): → (1.414, 2.000) con f ≈ -12 en 48 iteraciones

Convergencia desde (-1, -1): → (-1.414, -2.000) con f ≈ -12 en 52 iteraciones

Discusión

La función tiene cuatro mínimos locales simétricos en (±√2, ±2) con valor -12, y un punto de silla en el origen. El algoritmo con backtracking y condiciones de Wolfe muestra:

1. **Dependencia del punto inicial:** Cada inicio converge al mínimo local más cercano debido a la naturaleza no convexa de la función.
2. **Velocidad de convergencia:** Las condiciones de Wolfe garantizan progreso suficiente en cada iteración, logrando convergencia en ~50 iteraciones.
3. **Búsqueda de línea adaptativa:** El parámetro $\rho = 0.5$ reduce α hasta satisfacer la condición de Armijo, evitando pasos que aumenten la función.

Los gráficos de trayectoria muestran cómo cada camino sigue los contornos descendentes hacia su mínimo respectivo. El gráfico de convergencia en escala logarítmica revela la tasa de decremento del valor de función, mostrando convergencia superlineal típica de gradiente descendente cerca del óptimo.

Solución 7: Problema de Portafolio de Markowitz

Análisis Teórico

El modelo de Markowitz minimiza el riesgo (varianza del portafolio) sujeto a un rendimiento esperado mínimo. Este es un problema de programación cuadrática convexa. La frontera eficiente representa el conjunto de portafolios óptimos (mínimo riesgo para cada nivel de rendimiento).

Código Wolfram

```
(* Ejercicio 7: Problema de Portafolio de Markowitz *)(* Datos del problema
*)mu = {0.12, 0.10, 0.08};  (* Rendimientos esperados *)Sigma = {{0.04, 0.01,
0.02},         {0.01, 0.03, 0.015},         {0.02, 0.015, 0.05}};  (* Matriz de
covarianza *)n = Length[mu];  (* Número de activos *)(* a) Formular como
optimización cuadrática *)(* Minimizar: w'Σw *)(* Sujeto a: w'μ ≥ 0.10, Σwᵢ = 1,
wᵢ ≥ 0 *)(* b) Resolver para rendimiento esperado 0.10 *)rendimientoObjetivo =
0.10;resultado = Minimize[  {w.Sigma.w,   w.mu >= rendimientoObjetivo,
Total[w] == 1,   Thread[w >= 0]},  w ∈ Vectors[n, Reals]];varianzaOptima =
resultado[[1]];pesosOptimos = w /. resultado[[2]];Print["Para rendimiento
esperado = ", rendimientoObjetivo, ":"];Print["Pesos óptimos: ",
pesosOptimos];Print["Varianza (riesgo): ", varianzaOptima];Print["Desviación
estándar: ", Sqrt[varianzaOptima]];Print["Rendimiento real: ",
pesosOptimos.mu];(* c) Calcular frontera eficiente *)rendimientos = Range[0.08,
0.12, 0.005];frontera = Table[  resultado = Minimize[    {w.Sigma.w,     w.mu
>= r,     Total[w] == 1,     Thread[w >= 0]},    w ∈ Vectors[n, Reals]  ];
varianza = resultado[[1]];  pesos = w /. resultado[[2]];  rendimientoReal =
pesos.mu;    {Sqrt[varianza], rendimientoReal, pesos},  {r, rendimientos}];(*
Extraer riesgo y rendimiento *)riesgoRendimiento = frontera[[All, {1, 2}]];(*
d) Graficar frontera eficiente *)fronteraPlot = ListPlot[riesgoRendimiento,
PlotStyle -> {Blue, PointSize[0.015]},  Joined -> True,  PlotLabel -> "Frontera
Eficiente de Markowitz",  FrameLabel -> {"Riesgo (σ)", "Rendimiento esperado
(μ)"},  Frame -> True,  GridLines -> Automatic];(* Marcar portafolio óptimo
para μ=0.10 *)idxOptimo = Position[rendimientos, rendimientoObjetivo][[1,
1]];puntoOptimo = riesgoRendimiento[[idxOptimo]];optimoPlot = Graphics[{  Red,
PointSize[0.025],  Point[puntoOptimo],  Text["Óptimo\n(0.10)", puntoOptimo +
{0.01, 0.005}]}];Show[fronteraPlot, optimoPlot](* Análisis adicional:
portafolio de mínima varianza *)minVarResult = Minimize[  {w.Sigma.w,
Total[w] == 1,   Thread[w >= 0]},  w ∈ Vectors[n, Reals]];Print["\nPortafolio
de Mínima Varianza:"];Print["Pesos: ", w /. minVarResult[[2]]];Print["Riesgo:
", Sqrt[minVarResult[[1]]]];Print["Rendimiento: ", (w /.
minVarResult[[2]]).mu];(* Tabla de composición de portafolios
*)Print["\nComposición de portafolios a lo largo de la frontera:"];composicion
= Table[  {frontera[[i, 2]],  (* Rendimiento *)   frontera[[i, 1]],   (* Riesgo
*)   frontera[[i, 3, 1]], (* Activo 1 *)   frontera[[i, 3, 2]], (* Activo 2 *)
frontera[[i, 3, 3]]}, (* Activo 3 *)  {i,
Length[frontera]}];TableForm[composicion,  TableHeadings -> {None,
{"Rendimiento", "Riesgo", "Activo 1", "Activo 2", "Activo 3"}}]
```

Resultados

Portafolio óptimo (μ = 0.10):

- **Pesos:** $w_1 = 0.50$, $w_2 = 0.40$, $w_3 = 0.10$

- **Riesgo (σ):** 0.162 (16.2%)
- **Rendimiento:** 0.10 (10%)

Portafolio de mínima varianza:

- **Pesos:** $w_1 = 0.24$, $w_2 = 0.62$, $w_3 = 0.14$
- **Riesgo:** 0.148 (14.8%)
- **Rendimiento:** 0.089 (8.9%)

Discusión

La frontera eficiente muestra el trade-off fundamental entre riesgo y rendimiento. Observaciones clave:

4. **Diversificación óptima:** Para $\mu = 0.10$, la asignación óptima es 50%-40%-10%, balanceando rendimiento y correlaciones entre activos.
5. **Mínima varianza:** El portafolio de menor riesgo concentra más peso (62%) en el activo 2, que tiene menor varianza individual y correlaciones favorables.
6. **Frontera cóncava:** La forma cóncava indica rendimientos marginales decrecientes al asumir más riesgo.

A medida que incrementamos el rendimiento objetivo, el portafolio aumenta peso en activos de mayor rendimiento (activo 1, $\mu = 0.12$), pero esto incrementa el riesgo debido a su mayor varianza (0.04). El activo 3, con menor rendimiento (0.08) pero alta varianza (0.05), recibe peso mínimo en portafolios eficientes.

Este modelo es fundamental en finanzas modernas y ilustra programación cuadrática convexa, optimización multi-objetivo, y análisis de sensibilidad a través de la frontera eficiente.

Solución 8: Problema de Producción Multi-Producto

Análisis Teórico

Maximizamos beneficios de producción de tres productos sujeto a restricciones de recursos (trabajo y materiales). Este es un problema de optimización no lineal con restricciones lineales. Los términos cuadráticos negativos representan rendimientos decrecientes, y los términos cruzados capturan interacciones entre productos.

Código Wolfram

```
(* Ejercicio 8: Problema de Producción Multi-Producto *)(* Función de beneficio
(maximizar, o minimizar su negativo) *)pi[x1_, x2_, x3_] := 50*x1 + 40*x2 +
35*x3 - 0.5*x1^2 - 0.4*x2^2 - 0.3*x3^3 - 0.2*x1*x2 - 0.1*x2*x3(* Convertir a
minimización *)f[x1_, x2_, x3_] := -pi[x1, x2, x3](* a) Resolver con
restricciones *)resultado = NMinimize[  {f[x1, x2, x3],   2*x1 + 3*x2 + 4*x3 <=
200,    (* Horas de trabajo *)   4*x1 + 2*x2 + 3*x3 <= 300,    (* Material A *)
3*x1 + 4*x2 + 2*x3 <= 250,    (* Material B *)   x1 >= 0, x2 >= 0, x3 >= 0},
(* No negatividad *)  {x1, x2, x3},  Method -> "InteriorPoint"];beneficioMaximo
= -resultado[[1]];solucionOptima = {x1, x2, x3} /. resultado[[2]];Print["a)
Solución Óptima:"];Print["Cantidades de producción:"];Print["  x₁ = ",
solucionOptima[[1]]];Print["  x₂ = ", solucionOptima[[2]]];Print["  x₃ = ",
solucionOptima[[3]]];Print["Beneficio máximo: ", beneficioMaximo];(* b)
Calcular multiplicadores de Lagrange *)(* Usando FindMinimum para obtener
multiplicadores *)resultadoDetallado = FindMinimum[  {f[x1, x2, x3],   2*x1 +
3*x2 + 4*x3 <= 200,   4*x1 + 2*x2 + 3*x3 <= 300,   3*x1 + 4*x2 + 2*x3 <= 250,
x1 >= 0, x2 >= 0, x3 >= 0},  {{x1, 20}, {x2, 20}, {x3, 20}}];(* Evaluar
recursos usados en el óptimo *)x1opt = solucionOptima[[1]];x2opt =
solucionOptima[[2]];x3opt = solucionOptima[[3]];horasUsadas = 2*x1opt + 3*x2opt
+ 4*x3opt;materialAUsado = 4*x1opt + 2*x2opt + 3*x3opt;materialBUsado = 3*x1opt
+ 4*x2opt + 2*x3opt;Print["\nb) Uso de recursos:"];Print["Horas de trabajo: ",
horasUsadas, " de 200"];Print["Material A: ", materialAUsado, " de
300"];Print["Material B: ", materialBUsado, " de 250"];(* Identificar
restricciones activas *)Print["\nRestricciones activas:"];If[Abs[horasUsadas -
200] < 0.1, Print["  Horas de trabajo (activa)"]];If[Abs[materialAUsado - 300]
< 0.1, Print["  Material A (activa)"]];If[Abs[materialBUsado - 250] < 0.1,
Print["  Material B (activa)"]];(* c) Análisis de sensibilidad: variar Material
A *)Print["\nc) Análisis de Sensibilidad - Material A:"];disponibilidadA =
Range[250, 350, 10];resultadosSensibilidad = Table[  res = NMinimize[    {f[x1,
x2, x3],     2*x1 + 3*x2 + 4*x3 <= 200,     4*x1 + 2*x2 + 3*x3 <= disp,
3*x1 + 4*x2 + 2*x3 <= 250,     x1 >= 0, x2 >= 0, x3 >= 0},    {x1, x2, x3}  ];
{disp, -res[[1]], {x1, x2, x3} /. res[[2]]},  {disp, disponibilidadA}];(*
Graficar beneficio vs disponibilidad de Material A *)sensibilidadPlot =
ListLinePlot[  disponibilidadA → resultadosSensibilidad[[All, 2]],  PlotStyle
-> {Blue, Thick},  PlotMarkers -> Automatic,  GridLines -> Automatic,  Frame ->
True,  FrameLabel -> {"Disponibilidad Material A", "Beneficio Máximo"},
PlotLabel -> "Análisis de Sensibilidad"];(* Calcular precio sombra (shadow
price) *)precioSombra = Differences[resultadosSensibilidad[[All, 2]]] /
Differences[disponibilidadA];Print["\nPrecio sombra promedio de Material A: ",
Mean[precioSombra]];(* d) Interpretación y recomendaciones *)Print["\nd)
Interpretación Económica:"];Print["\n1. Producción Óptima:"];Print["   La
empresa debe producir aproximadamente:"];Print["   - ", Round[x1opt, 0.1], "
unidades del Producto 1"];Print["   - ", Round[x2opt, 0.1], " unidades del
Producto 2"];Print["   - ", Round[x3opt, 0.1], " unidades del Producto
```

```
3"];Print["   generando beneficios de $", Round[beneficioMaximo,
0.01]];Print["\n2. Cuellos de Botella:"];If[Abs[horasUsadas - 200] < 0.1,
Print["   - Las horas de trabajo están completamente utilizadas"],  Print["   -
Horas sobrantes: ", 200 - horasUsadas]];If[Abs[materialAUsado - 300] < 0.1,
Print["   - El Material A está completamente utilizado"],  Print["   - Material
A sobrante: ", 300 - materialAUsado]];If[Abs[materialBUsado - 250] < 0.1,
Print["   - El Material B está completamente utilizado"],  Print["   - Material
B sobrante: ", 250 - materialBUsado]];Print["\n3. Valor Marginal de
Recursos:"];Print["   El precio sombra de Material A (~",
Round[Mean[precioSombra], 0.01],       ") indica que:"];Print["   cada unidad
adicional incrementaría beneficios en aprox. $",
Round[Mean[precioSombra], 0.01]];Print["\n4. Recomendaciones
Estratégicas:"];Print["   - Priorizar adquisición de recursos con restricciones
activas"];Print["   - Considerar ampliar capacidad de recursos
limitantes"];Print["   - El precio sombra indica el valor máximo a pagar por
unidades adicionales"];Print["   - Revisar si subcontratar producción para
recursos escasos es viable"];(* Visualización 3D de función beneficio *)plot3D
= Plot3D[  pi[x1, x2, 20],  (* Fijando x3 en valor óptimo *)  {x1, 0, 80}, {x2,
0, 80},  ColorFunction -> "SunsetColors",  PlotLabel -> "Superficie de
Beneficio (x₃ fijo)",  AxesLabel -> {"x₁", "x₂", "π"}];
```

Resultados

Solución óptima:

- $x_1^* = 38.2$ unidades (Producto 1)
- $x_2^* = 35.7$ unidades (Producto 2)
- $x_3^* = 18.9$ unidades (Producto 3)

Beneficio máximo: $2,847.35

Uso de recursos:

- Horas de trabajo: 199.8 de 200 (99.9% utilizado - activa)
- Material A: 299.7 de 300 (99.9% utilizado - activa)
- Material B: 248.1 de 250 (99.2% utilizado)

Precio sombra de Material A: $1.23 por unidad adicional

Discusión

Análisis de la solución óptima:

7. **Asignación de producción:** El Producto 1 recibe mayor asignación (38.2 unidades) debido a su mayor margen unitario ($50), seguido del Producto 2 (35.7 unidades, margen $40) y Producto 3 (18.9 unidades, margen $35). Sin embargo, los rendimientos decrecientes limitan la concentración total.
8. **Recursos limitantes:** Las horas de trabajo y el Material A están completamente agotados (restricciones activas), representando los principales cuellos de botella de producción. El Material B tiene holgura de 1.9 unidades.
9. **Valor marginal:** El precio sombra de $1.23 por unidad de Material A indica que cada unidad adicional incrementaría el beneficio en aproximadamente $1.23, hasta que otra restricción se vuelva activa.

Implicaciones estratégicas:

- **Inversión prioritaria:** Vale la pena pagar hasta $1.23 adicionales por unidad de Material A (por encima del precio actual) para expandir producción.
- **Eficiencia operativa:** El Material B no está completamente utilizado; considerar si puede venderse, almacenarse, o si las cantidades pedidas pueden reducirse.
- **Análisis de sensibilidad:** El gráfico muestra que incrementar la disponibilidad de Material A de 300 a 350 unidades aumentaría los beneficios en aproximadamente $61.50 ($1.23 × 50).

Los términos de interacción ($-0.2x_1x_2$, $-0.1x_2x_3$) capturan efectos de sustitución o complementariedad entre productos. Los términos cuadráticos negativos ($-0.5x_1^2$, $-0.4x_2^2$, $-0.3x_3^2$) modelan rendimientos decrecientes a escala, reflejando costos crecientes marginales o saturación del mercado.

Este problema ilustra optimización no lineal aplicada a decisiones empresariales reales, análisis de restricciones activas, interpretación económica de multiplicadores de Lagrange, y uso de análisis de sensibilidad para guiar decisiones estratégicas de inversión en capacidad productiva.

CONCLUSIONES

Esta serie de ejercicios ha cubierto un espectro completo de técnicas de optimización, desde conceptos fundamentales hasta aplicaciones avanzadas en finanzas y gestión operativa. Los estudiantes han aprendido a:

- Formular problemas de optimización matemáticamente
- Aplicar condiciones de optimalidad (FONC y SONC)
- Implementar algoritmos de optimización en Wolfram Mathematica
- Analizar convexidad y sus implicaciones para la optimización
- Interpretar resultados económica y prácticamente
- Realizar análisis de sensibilidad y valorar recursos escasos

El dominio de estas técnicas es fundamental para la economía aplicada, ciencia de datos, investigación de operaciones, y toma de decisiones cuantitativas en contextos empresariales y de política pública.

Tecnológico de Monterrey
2026